CRYPTGO

25JE0716

Project Overview

Cryptgo is a website where users can upload their files and keep them secure. The main idea of the project was to ensure that uploaded files remained private and could not be read by anyone else, including the server. To achieve this, files were encrypted on the user's device before being uploaded to the platform.

Only the same user, or users with whom the file was shared, could later decrypt and access the file. The platform included a user login system along with an additional security layer in the form of multi-factor authentication (MFA), where users entered a time-based code generated by the system.

The project also explored secure file sharing, allowing users to share their encrypted files with others while maintaining privacy and security.

Work completed so far

While creating CryptGo, I worked on building the main components required for a secure file storage platform.

On the backend, I developed the server using **Python Flask** and connected it to a **PostgreSQL** database. A separate database connection module was created and tested to ensure stable communication between the application and the database. This helped establish a reliable backend structure for handling user-related data.

I implemented a **user authentication system** that supports signup and login functionality. User passwords are securely hashed before being stored in the database, ensuring that sensitive credentials are never saved in plain text. This step helped me understand real-world authentication and security practices.

To align with security best practices, sensitive information such as database credentials and secret keys was managed using **environment variables** instead of being directly written into the source code. This ensured that confidential data remained protected and was not exposed in the repository.

The project also explored the **design of client-side encryption for file storage**. The idea was to ensure that files are encrypted on the user's device before being uploaded, so that the server never has access to the original file contents. While the project mainly

focuses on understanding and structuring this encryption workflow, it helped in learning how end-to-end encrypted systems are designed.

On the frontend, I learned the basics of **HTML and CSS** and created a simple interface to explain the project and allow user interaction. Using **JavaScript**, I connected the frontend to the backend APIs and tested the complete authentication flow. Error cases such as duplicate user registration were handled and tested to ensure proper backend validation.

Overall, the project helped me learn how authentication works, how files can be kept secure using encryption, and how the frontend and backend are connected in a web application.

Challenges faced

One major challenge was learning frontend technologies like HTML, CSS, and JavaScript from scratch. Initially, it was difficult to understand how the frontend communicates with the backend.

I also faced issues related to **CORS errors** when connecting the frontend to the backend, which took time to understand and fix. I took the help of LLMs to understand the issue and found the solution to the problem.

Another challenge was setting up **PostgreSQL** and understanding how database connections and queries work in a real application. Debugging backend errors and understanding error messages was challenging but helped me learn better.

One of the main challenges during the later stages of the project was designing the overall **structure** of a secure system rather than just writing code.

Ensuring that sensitive information like **passwords and secret keys** was never exposed in the codebase was a challenge faced in the later phase. Handling **environment variables** and configuring the application to run securely on different systems was also challenging. Since sensitive credentials could not be included in the repository, extra care was needed to document the setup clearly while keeping the project secure.

Finally, balancing the project scope with the available time was a challenge. Some **advanced features** (like sending the OTP to the user's email, role based access, expiring links, folder system, file preview) were intentionally kept at a conceptual or partial level so that the core functionality could remain stable and well-structured.

Future Scope

The following features were identified as potential improvements for CryptGo but were not implemented in the current version of the project due to time and scope constraints. These features can be added in future iterations to enhance usability and security.

- **Role-Based Access Control:**
  Each file can be assigned roles such as owner, editor, and viewer, where different roles provide different access rights for viewing, editing, or managing files.
- **Folder Organization:**
  Users can be given the ability to create folders to organize their files, instead of storing all files in a single list.
- **File Preview:**
  For supported file types such as text files or images, the platform can display a small preview without requiring the user to download the entire file.
- **Expiring Share Links:**
  For additional security, users can generate shareable links that automatically expire after a selected duration (for example, 10 minutes or 1 day). Once expired, the link would no longer allow access to the file.

Research / concepts learned

During the development of CryptGo, I gained practical understanding of several core concepts related to secure web applications.

I learned the basics of **HTML**, **CSS**, **JavaScript** in the frontend, **Python Flask** in the backend, **PostgreSQL** in the database required for creating any webpage

I learned how **authentication systems** are designed, including secure handling of user credentials and password hashing to protect sensitive information. This helped me understand why plain-text passwords should never be stored in real applications.

I explored the concept of **end-to-end encryption**, where data is encrypted on the client side before being sent to the server. This allowed me to understand how user data can remain private even from the server itself, which is an important idea in modern secure systems.

The project also helped me understand how **frontend and backend components communicate** using APIs and JSON, and how responsibilities are divided between the client and the server in a web application.

I learned the importance of **environment variables** and configuration management to keep secrets such as database credentials and keys out of the codebase. This introduced me to real-world security practices used in professional development.

Additionally, I gained experience in **database design and interaction** using PostgreSQL, including handling user data and enforcing basic validation and constraints.

Overall, the project helped me develop a better understanding of secure system design, application architecture, and practical security considerations in web development.